

회전 / 직진 튜닝 가이드

기기마다 모터 편차·마찰·바퀴 그림이 달라서 같은 코드라도 회전각·직진성이 조금씩 다릅니다. 이 문서는 어떤 증상에 어떤 값을 만져야 하는지 정리합니다.

핵심 원칙: 거동(behavior) 튜닝은 거의 전부 로봇별 설정 파일 한 곳에서 합니다. 로직([Controller.cpp](#))은 건드리지 않고 매크로 값만 바꿔서 재컴파일/업로드하면 됩니다. 각 매크로 위 주석에 조정 방향이 적혀 있습니다.

[듀얼 로봇 빌드] 튜닝 값은 이제 로봇별 [Settings_robot1.h](#) / [Settings_robot2.h](#) 에 있고, [Settings.h](#) 는 [ROBOT_ID](#) 선택터(27줄)입니다. 자기 로봇의 [Settings_robot<ID>.h](#) 를 편집할 것. 아래 라인 앵커는 robot1([Settings_robot1.h](#)) 기준입니다. robot2 는 같은 매크로 이름을 갖지만 기본값이 일부 다릅니다. [ROBOT_ID](#) 전환: [Settings.h](#) 에서 [#define ROBOT_ID 1](#) → [2](#) 로 바꾸거나, 빌드 플래그 [-DROBOT_ID=2](#) 를 사용하세요.

회전은 모두 **사다리꼴 가감속(RampTurn)**으로 동작하고, 라인트레이싱은 **PD 제어**, 화물 적재 여부 ([_hasPayload](#))에 따라 별도 [_CARGO](#) 값으로 전환됩니다. 옛 버전의 수동 킥스타트 회전 / P 제어 / [Power-20](#) 감속은 더 이상 없습니다.

1. 회전 함수 3종 — 구조

세 회전 함수([TurnHalf](#), [PivotTurnLeft](#), [PivotTurnRight](#))는 모두 공통 프리미티브 [RampTurn](#)을 호출합니다. [RampTurn](#) 은 사다리꼴 속도 프로파일로 돕니다:

가속([TURN_START_PWM](#) → [cruise](#)) → 정속([cruise](#), [holdMs](#)) → 감속([cruise](#) → [TURN_START_PWM](#)) → 정지 → 안정화([TURN_SETTLE_MS](#))

- 정속 PWM([cruise](#)) = 회전 속도. 빠르게 돌릴수록 관성 overshoot 위험.
- [holdMs](#)(정속 시간) = 사실상 회전각을 결정 (회전각 ≈ PWM × 시간).
- 가감속 구간도 회전을 보탭니다 — [TURN_RAMP_STEPS/TURN_RAMP_STEP_MS](#) 를 바꾸면 회전각이 같이 변하므로 *[_DELAY_MS](#) 로 재보정 필요.

[TurnHalf](#) (≈180°)

제자리 회전(왼쪽 후진 + 오른쪽 전진, 양 바퀴 같은 PWM). [Controller.cpp:793](#).

[PivotTurnLeft](#) / [PivotTurnRight](#) (≈90°)

강쪽(바깥 바퀴) / 약쪽(안쪽 바퀴) PWM 을 비대칭으로 줘서 90° 회전. 두 PWM 차이가 클수록 돌면서 앞으로 쏠리는 완만한 호, 같으면 순수 제자리 회전. [Controller.cpp:802](#) / [:814](#).

화물 적재 시 ([_CARGO](#))

[LifterUp\(\)](#) 으로 팔레트를 들면 [_hasPayload=true](#) 가 되고, 이후 모든 회전이 [_CARGO](#) 변형 상수(더 잘게 쪼갠 가감속 + 낮춘 PWM + 보충한 delay)로 전환되어 팔레트 슬라이드/관성 흔들림을 줄입니다. [LifterDown\(\)](#) 이 다시 일반 거동으로 되돌립니다.

직진 / 라인트레이싱

PD 제어(K_p , K_d)가 좌/우 PWM 을 조정하고, base PWM 은 **직진 모션 프로파일**(가감속 램프 슬루)이 만듭니다 — 출발 킥 (**DRIVE_START_PWM**) → 가속 → 정속(**MOTOR_POWER**, 기본 140) → 런 마지막 칸 감속(**DRIVE_END_PWM**) → 마지막 교차로 통과 → 정지. 중간 교차로는 정속 무정지 통과. **drive()** 안에서 EEPROM 모터 캘리브가 곁해집니다(§4.4).

2. 핵심 튜닝 값 한눈에

전부 **Settings_robot1.h**에 있습니다 (robot2 는 **Settings_robot2.h** — 매크로 이름 동일, 값 일부 상이).

값	위치	기본값	설명
MOTOR_POWER / _CARGO	Settings_robot1.h:87	140 / 140	정속(cruise) 상한 PWM (일반/화물)
DRIVE_START_PWM / _CARGO	Settings_robot1.h:81	90 / 90	런 출발 킥 PWM
DRIVE_END_PWM / _CARGO	Settings_robot1.h:93	70 / 70	끝(노드) 감속 바닥 = 마지막 교차로 통과 PWM (구 CROSSING_PASS_POWER)
DRIVE_ACCEL_MS / _CARGO	Settings_robot1.h:96	1000	START→cruise 가속 시간(ms)
DRIVE_DECEL_MS / _CARGO	Settings_robot1.h:99	130	cruise→END 감속 시간(ms)
DRIVE_BRAKE_CELLS	Settings_robot1.h:103	1	런 끝 몇 칸 전부터 감속 시작
PRECISE_REALIGN_ENABLE	Settings_robot1.h:162	0	y=0/y=7 도착 시 후진→전진 정렬 dance on/off (0=끔)
REALIGN_BACKUP_MS / _CREEP_MS	Settings_robot1.h:166	240 / 120	정렬 dance 후진 / 전진크리프 시간
CROSSING_PASS_MS	Settings_robot1.h:174	100	교차로 통과 블라인드 윈도우(ms) — 중복 카운트 방지
PID_KP / PID_KD	Settings_robot1.h:114	0.09 / 0.24 (식)	PD 비례 / 미분 항
PID_MAX_CORRECTION	Settings_robot1.h:123	25.0f	조향 보정 saturation
LINEDETECT_NORM_MIN	Settings_robot1.h:141	700	교차로 검출 임계값 (정규화 0~1000 기준)
LINEDETECT_RAW_FALLBACK	Settings_robot1.h:146	730	캘리브 무효 시 폴백 raw 임계값
OBSTACLE_THRESHOLD / _SIDE	Settings_robot1.h:150	700 / 700	IR 장애물 임계값 (중앙 / 좌·우)
TurnHalf PWM / DELAY (일반/화물)	Settings_robot1.h:255	170/330, 120/410	180° 속도/각도
PivotLeft STRONG/WEAK/DELAY	Settings_robot1.h:265	170/120/130	좌회전 속도/각도 (화물 120/70/225)

값	위치	기본값	설명
PivotRight STRONG/WEAK/DELAY	Settings_robot1.h:275	170/120/140	우회전 속도/각도 (화물 120/70/230)
TURN_RAMP_STEPS / _STEP_MS	Settings_robot1.h:220	8 / 12	가감속 분할/스텝시간 (부드러움). 화물 14/10
TURN_START_PWM	Settings_robot1.h:222	90	가감속 시작/끝 PWM (최소 회전 속도)
TURN_SETTLE_MS	Settings_robot1.h:236	150	회전 후 차체 안정화 대기
SERVO_DOWN / SERVO_UP	Settings_robot1.h:180	20° / 80°	리프터 내림/올림 각도
SERVO_STEP_DEG / _STEP_MS	Settings_robot1.h:188	2 / 15	리프터 부드러운 슬루
_motorCalibL/R	EEPROM (+8/+12)	float	직진 보정 (별도 캘리브 스케치)

디버그 토글: [DEBUG_TRACE](#)([Settings_robot1.h:132](#), 기본 0),
[DEBUG_APPROACH_TONE](#)([Settings_robot1.h:107](#), 기본 1),
[DEBUG_TURN_PAUSE_MS](#)([Settings_robot1.h:241](#), 기본 0). 실주행/대회 전 §8 참고해 정리.

3. 증상 → 만질 값

증상	우선 조정	어디서
좌회전 부족 (90° 못 채움)	PIVOT_LEFT_DELAY_MS ↑ (+10)	§4.1
좌회전 과회전	PIVOT_LEFT_DELAY_MS ↓	§4.1
우회전만 다름	PIVOT_RIGHT_DELAY_MS 따로 조정	§4.1
화물 들 때만 어긋남	*_DELAY_MS_CARGO 쪽만 조정	§4.1
180° 어긋남	TURNHALF_DELAY_MS ↑/↓ (±30)	§4.1
회전 시작 멈칫 (출발 안 함)	TURN_START_PWM ↑	§4.1
회전 시작/끝이 거칠고 덜컹임	TURN_RAMP_STEPS ↑	§4.1
회전 한 번이 너무 오래 걸림	TURN_RAMP_STEP_MS ↓	§4.1
화물 회전이 90° 넘게 뒤편 (부드러움 유지)	TURN_RAMP_STEP_MS_CARGO ↓	§4.1
직진 한쪽으로 휨	_motorCalibL/R (별도 캘리브 스케치)	EEPROM
라인 위 갈지자/진동	PID_KD ↓ 또는 PID_KP ↓	§4.4
곡선에서 라인 따라가는 게 느림(lag)	PID_KP ↑	§4.4
곡선/급이탈에서 바깥으로 흘러나감	PID_MAX_CORRECTION ↑	§4.4
직선에서 좌우 지그재그	PID_MAX_CORRECTION ↓	§4.4

증상	우선 조정	어디서
라인 인식 누락	LINEDETECT_NORM_MIN ↓	§4.4
라인 외 잡음에 트립	LINEDETECT_NORM_MIN ↑	§4.4
노드 지나치고 멈춤(overshoot)	DRIVE_END_PWM ↓ 또는 DRIVE_BRAKE_CELLS 2	§4.4
긴 직선이 느림	MOTOR_POWER ↑	§4.4
출발 덜컹임 / 가속 급함	DRIVE_ACCEL_MS ↑	§4.4
노드 직전 감속이 약함	DRIVE_DECEL_MS ↓ (급제동)	§4.4
교차로 통과 못 넘김 / 과하게 넘김	CROSSING_PASS_MS ↑/↓	§4.5
y=0/y=7 도착 정렬이 너무 느림	PRECISE_REALIGN_ENABLE 0 (dance 끄)	§4.5
정렬 dance 가 선을 지나침/못 미침	REALIGN_BACKUP_MS ↓/↑	§4.5
화물 적재 주행이 너무 빠름	MOTOR_POWER_CARGO ↓	§4.4
장애물 오감지 (없는데 트립)	OBSTACLE_THRESHOLD ↓	§4.4
장애물 미감지	OBSTACLE_THRESHOLD ↑	§4.4
측면 벽/라인에 장애물 오감지	OBSTACLE_THRESHOLD_SIDE ↓	§4.4
리프터가 화물에 안 닿음	SERVO_UP ↑	§4.3
리프터가 바닥 긁음	SERVO_DOWN ↓	§4.3

4. 조정 값 — Settings_robot*.h 블록별

값은 전부 [Settings_robot1.h\(robot1\)](#) 또는 [Settings_robot2.h\(robot2\)](#)에 모여 있습니다. 아래는 각 블록의 실제 내용입니다. 주석의 조정 방향을 보고 숫자만 바꾸세요. 로직(**Controller.cpp**)은 건드릴 필요 없습니다.

4.1 회전 — 가감속(공통) + 회전별 PWM/DELAY

가감속 공통 ([Settings_robot1.h:214](#)):

```
#define TURN_RAMP_STEPS      8      // 가감속 분할 수. ↑ 더 부드럽게 / ↓ 빠릿하게
#define TURN_RAMP_STEP_MS    12     // 한 스텝 유지 ms. ↑ 완만하게 / ↓ 빨리 끝냄
#define TURN_START_PWM       90     // 가감속 시작/끝 PWM(최소 회전 속도). 출발 멈칫 ↑
/ 거칠면 ↓
#define TURN_RAMP_STEPS_CARGO 14     // 화물: 부드러움은 STEPS 에서 나옴(크게 유지)
#define TURN_RAMP_STEP_MS_CARGO 10   // 화물: 과회전은 STEP_MS 에서 - 90° 넘으면
↓
#define TURN_START_PWM_CARGO  90
#define TURN_SETTLE_MS        150   // 회전 후 안정화 대기. 인식 흔들리면 ↑ / 굼뜨면 ↓
```

회전별 속도(PWM)/각도(DELAY) ([Settings_robot1.h:249](#)):

```
// TurnHalf 180° (양 바퀴 같은 PWM)
#define TURNHALF_PWM 170 // [일반] 속도
#define TURNHALF_DELAY_MS 330 // [일반] 각도 - 못 채움 ↑ / 넘게 뚫 ↓
#define TURNHALF_PWM_CARGO 120 // [화물] 속도
#define TURNHALF_DELAY_MS_CARGO 410 // [화물] 각도

// PivotTurnLeft 90° (STRONG=바깥, WEAK=안쪽)
#define PIVOT_LEFT_STRONG_PWM 170 // [일반] 바깥 바퀴 속도
#define PIVOT_LEFT_WEAK_PWM 120 // [일반] 안쪽 바퀴 속도(차 ↑ → 호가 커짐)
#define PIVOT_LEFT_DELAY_MS 145 // [일반] 각도
#define PIVOT_LEFT_STRONG_PWM_CARGO 120 // [화물]
#define PIVOT_LEFT_WEAK_PWM_CARGO 70
#define PIVOT_LEFT_DELAY_MS_CARGO 225

// PivotTurnRight 90° (구조 동일, 좌/우 편차로 값 다를 수 있음)
#define PIVOT_RIGHT_STRONG_PWM 170
#define PIVOT_RIGHT_WEAK_PWM 120
#define PIVOT_RIGHT_DELAY_MS 155 // 좌(145)와 별도 - 우만 어긋나면 여
기만
#define PIVOT_RIGHT_STRONG_PWM_CARGO 120
#define PIVOT_RIGHT_WEAK_PWM_CARGO 70
#define PIVOT_RIGHT_DELAY_MS_CARGO 230
```

△ `TURN_RAMP_STEPS/STEP_MS/START_PWM` 을 바꾸면 가감속 구간이 회전을 보태는 양이 달라져 회전각이 변합니다. 그 세 값을 손대면 위 `*_DELAY_MS` 로 90°/180° 를 다시 맞추세요. (실제로 가감속 도입 후 `TurnHalf` 가 더 돌아서 `TURNHALF_DELAY_MS` 를 450→330 으로 낮춘 상태.)

참고로 회전 본체는 손댈 일이 없지만, 동작 이해용 함수는 `RampTurn` / `TurnHalf` / `PivotTurnLeft` / `PivotTurnRight`.

4.2 디버그 — 회전각 측정

```
#define DEBUG_TURN_PAUSE_MS 0 // >0 으로 두면 매 회전 후 그만큼 정지 → 각도기로 측
정
// △ 실주행 전 반드시 0
```

4.3 리프터 서보

```
#define SERVO_DOWN (90 - 70) // 내림 = 20°. 바닥 굽으면 ↓
#define SERVO_UP (180 - 150) // 올림 = 80°. 화물 안 닿으면 ↑
#define SERVO_DEF SERVO_DOWN // 부팅 위치
#define SERVO_STEP_DEG 2 // 스텝당 각도(작을수록 부드럽고 느림)
#define SERVO_STEP_MS 15 // 스텝 간 대기 ms
#define SERVO_SETTLE_MS 120 // 도달 후 detach 전 안착 대기
```

리프터는 **LifterMove**가 목표각까지 **SERVO_STEP_DEG** 씩 단계적으로 슬루해 부드럽게 올라갑니다. 이동 총시간 $\approx$ (각도차 / **SERVO_STEP_DEG**) $\times$ **SERVO_STEP_MS**.

4.4 직진 — PD 제어 + 감속 + 검출/장애물 임계값

PD 제어 원리·예시·튜닝 값은 [PD제어_가이드.md](#) 참고. 여기는 설정값 레퍼런스.

PD 제어 ([Settings_robot1.h:114](#)):

```
#define PID_KP    (0.05f + (+0.8 * 0.05f)) // = 0.09f. 비례. 곡선 lag ↑ / 진동 ↓ (±10% 단위 조절식)
#define PID_KD    (0.3f + (-0.2 * 0.3f)) // = 0.24f. 미분(진동 억제). 진동 ↓
#define PID_MAX_CORRECTION 25.0f // 조향 보정 saturation (좌우 PWM 차 최대 2×).

// 바깥으로 흘러나감 ↑ / 직선 지그재
그 ↓
```

직진 모션 프로파일 ([Settings_robot1.h:81](#)) — base PWM 을 매 루프 목표로 **가감속을 제한 슬루해서** 사다리꼴/삼각형 속도 프로파일을 만듭니다 (엔코더 없음 → PWM≈속도 근사). 전 항목 [일반]/[화물(**CARGO**)] 분리:

```
#define DRIVE_START_PWM 90 // 출발 킁(정지마찰 위). 화물:
DRIVE_START_PWM_CARGO
#define MOTOR_POWER 140 // 정속(cruise) 상한. 화물:
MOTOR_POWER_CARGO
#define DRIVE_END_PWM 70 // 끝(노드) 감속 바닥 = 마지막 교차로 통과 PWM. 화물: DRIVE_END_PWM_CARGO
#define DRIVE_ACCEL_MS 1000 // START→cruise 가속 시간(ms). 화물:
DRIVE_ACCEL_MS_CARGO
#define DRIVE_DECEL_MS 130 // cruise→END 감속 시간(ms). 화물:
DRIVE_DECEL_MS_CARGO
#define DRIVE_BRAKE_CELLS 1 // 런 끝 N칸 전부터 감속 (일반/화물 공통)
```

옛 **CROSSING_APPROACH_***(사전 감속 스텝)은 이 프로파일로 대체·제거됐고, 옛 **CROSSING_PASS_POWER** 는 **DRIVE_END_PWM** 으로 이름이 통일됐습니다.

동작 — 같은 heading 으로 이어지는 직선 = "런(run)"이고, 런 단위로 프로파일이 적용됩니다:

```
0(이전 노드) → DRIVE_START_PWM(출발 킁) → [가속 DRIVE_ACCEL_MS] → MOTOR_POWER(정속)
→ 런 마지막 DRIVE_BRAKE_CELLS 칸에서 [감속 DRIVE_DECEL_MS] → DRIVE_END_PWM
→ 마지막 교차로 통과 → Stop/realign(0)
```

- 중간 교차로는 **정속 무정지 통과**(딥 없음). 감속은 런의 마지막 칸에서만.
- **7칸 직선** = 가속→정속→감속 사다리꼴. **1칸** = 정속 못 미치고 바로 감속(낮은 피크 삼각형 — 영상 패턴과 동일).

사용 예제:

목표	조정
긴 직선 더 빠르게	MOTOR_POWER ↑ (예 140→170). $base \pm correction < 255$ 와 PID_* 재확인
출발 덜컹임 / 가속 부드럽게	DRIVE_ACCEL_MS ↑ (예 250→1000)
노드 직전 더 세게 감속	DRIVE_DECEL_MS ↓ (급제동)
노드 오버슈트 / 마지막 교차로 놓침	DRIVE_END_PWM ↓ 또는 DRIVE_BRAKE_CELLS 2 (한 칸 더 일찍 감속)
화물만 살살	MOTOR_POWER_CARGO ↓, DRIVE_ACCEL_MS_CARGO ↑, DRIVE_END_PWM_CARGO ↓

라인/장애물 검출 ([Settings_robot1.h:141](#)):

```
#define LINEDETECT_NORM_MIN      700    // 교차로 검출 임계(정규화 0~1000). 누
락 ↓ / 잡음 ↑
#define LINEDETECT_CALIB_MIN_SPAN 100    // 흑-백 격차 < 이 값이면 캘리브 무효 →
폴백
#define LINEDETECT_RAW_FALLBACK  730    // 캘리브 무효 시 폴백 raw 임계(비상용)
#define OBSTACLE_THRESHOLD        700    // 중앙 IR < 이 값이면 장애물. 오감지
↓ / 미감지 ↑
#define OBSTACLE_THRESHOLD_SIDE  700    // 좌·우 IR. 측면 오감지 줄이려면 ↓
```

동작: 라인 검출은 `onLine()`이 흑/백 캘리브 기반 정규화 값(흰0~검1000)으로 판정 → 로봇·바닥 독립. 캘리브가 무효(격차 < `LINEDETECT_CALIB_MIN_SPAN`)면 raw 폴백. 장애물은 `CheckObstacle()`이 중앙+좌+우 3센서를 1회 디바운스 재확인 후 판정. PD 제어 본체는 `LineTrace()`.

4.5 정렬/통과 타이밍 + 정밀 정렬 토글

y=0(참고)/y=7(도시) 도착 시의 후진→전진 정렬 dance 와 교차로 통과 블라인드 원도 ([Settings_robot1.h:162](#)):

```
#define PRECISE_REALIGN_ENABLE    0    // y=0/y=7 도착 정렬 dance on/off
// 0 = dance 생략(잠깐 정지만) → 빠르지만
정렬 정확도 ↓
#define REALIGN_BACKUP_MS        240    // dance 후진 시간. 구동
PWM=MOTOR_POWER. 선 지나침 ↓ / 못 미침 ↑
#define REALIGN_CREEP_MS         120    // 후진 후 라인 재검출 전진 크리프. 구동
PWM=MOTOR_POWER-40
#define CROSSING_PASS_MS         100    // 교차로 통과 블라인드 원도(미샘플 → 중복 카운트 방지).
// 마지막 교차로는 DRIVE_END_PWM, 중간은
_drivePwm(정속)으로 통과
```

REALIGN_* / CROSSING_PASS_MS 는 예전 SPEED_SCALE 로 나눠 쓰던 코드 내 하드코딩 상수였고, SPEED_SCALE 제거와 함께 Settings 로 분리됐습니다. 모두 "이동 거리 = 구동 PWM × 시간" 이라, 짝이 되는 PWM(MOTOR_POWER / DRIVE_END_PWM)을 바꾸면 거리가 변하니 함께 재튜닝하세요(\$7).

5. 권장 튜닝 순서

1. EEPROM 모터·센서 캘리브 먼저

- 워크스페이스의 별도 스케치(`Calibration.ino`, `MotorCalibration.ino`)로 흑/백 raw 값과 모터 `calib` 을 EEPROM 240번지 이후에 저장.
- 본 스케치는 시작 시 `readData()` 로 읽어 사용 — 비어 있으면 직진/정규화가 망가짐 (§6).

2. base 속도 결정 (§4.4)

- `MOTOR_POWER(cruise) / MOTOR_POWER_CARGO`. 정하면 `DRIVE_END_PWM` (< cruise 유지), `base±correction<255`, PD, 정렬 dance 재점검 — §4.4 주석의 ↔ 함께 확인.

3. 직진성 점검 (§4.4)

- 라인 위 갈지자 없이 진행되는지. `PID_KD` → `PID_KP` → `PID_MAX_CORRECTION` 순으로.

4. 검출 임계값 (§4.4)

- `DEBUG_TRACE=1` 로 라인 위 `L_n/R_n` 값을 보고 `LINEDETECT_NORM_MIN` 을 그 사이로.

5. PivotTurn 좌·우 (§4.1)

- 직진 → 정지 → 회전 한 번 → 직진 시퀀스로 90° 확인. `DEBUG_TURN_PAUSE_MS` 로 각도 측정.
- 일반/화물(`_CARGO`) 각각 조정.

6. TurnHalf 180° (§4.1). ±30 ms 단위.

7. 직진 모션 프로파일 (§4.4) — `MOTOR_POWER(cruise)` 정하고

`DRIVE_ACCEL_MS/DRIVE_DECEL_MS/DRIVE_END_PWM/DRIVE_BRAKE_CELLS` 로 가감속·노드 overshoot 미세 조정.

8. 실주행 전 디버그 토글 정리 (§8).

6. EEPROM 캘리브레이션 상세

`readData()` 가 EEPROM 240번지부터 6개 값을 읽어옵니다:

Offset	Type	필드	의미
+0	int16	<code>_rightWhite</code>	오른쪽 바닥센서 흰색 raw
+2	int16	<code>_leftWhite</code>	왼쪽 바닥센서 흰색 raw
+4	int16	<code>_rightBlack</code>	오른쪽 바닥센서 흑색(라인) raw
+6	int16	<code>_leftBlack</code>	왼쪽 바닥센서 흑색 raw
+8	float	<code>_motorCalibR</code>	오른쪽 PWM 배수
+12	float	<code>_motorCalibL</code>	왼쪽 PWM 배수

- 모터 `calib` 값: 보통 0.95 ~ 1.05 범위. 1.0 = 그대로. 한쪽이 빠르면 그쪽 값을 낮춤.
- 센서 흑/백 값: `normalizeLeft/Right()` 가 0~1000 정규화에 사용. 흰 raw < 검은 raw 여야 정상. 뒤집혀 있으면 정규화가 한쪽으로 항상 클램프되어 PD 제어가 한쪽으로만 꺾이는 증상이 납니다.
- 안전장치: 흑-백 격차가 `LINEDETECT_CALIB_MIN_SPAN`(100)보다 작으면 캘리브 무효로 보고 raw 폴백 (`LINEDETECT_RAW_FALLBACK`)으로 검출 — EEPROM 초기화 보드에서 정규화가 항상 1000 되는 오작동 방지.
- 본 스케치 플래시는 캘리브 영역(240~255)을 지우지 않습니다. 단 스톱 펌웨어 플래시는 지울 수 있으니 캘리브 후엔 이 스케치만 올리도록 주의.

EEPROM 0~199 영역은 별도로 NavLog 디버그 버퍼가 사용됩니다 (§8). 캘리브 영역과 겹치지 않습니다.

캘리브 이력 (2026-05-24)

정규화 도입 직후 `R_n` 이 항상 0 으로 클램프되어 P 제어가 한쪽으로만 꺾이는 증상이 있었음. 원인은 흑/백 값이 뒤집힌 (`_black < _white`) 옛 EEPROM 값. 정상 복구 후:

	leftWhite	leftBlack	rightWhite	rightBlack	motorCalibL	motorCalibR
Before(옛)	397	256	402	268	0.980	1.000
After(정상)	320	912	423	928	0.980	1.000

7. 속도(PWM)와 타이밍의 결합 주의

예전엔 전역 `SPEED_SCALE` 매크로가 PWM(x)과 delay(÷)를 한 번에 스케일했지만, 항상 1.0(거동 영향 없음)이라 제거했습니다. 코드에 박혀 있던 시간 상수는 §4.5 의 `REALIGN_BACKUP_MS` / `REALIGN_CREEP_MS` / `CROSSING_PASS_MS` 로 분리됐습니다.

핵심은 "이동 거리 = 구동 PWM × 시간" 이라는 결합입니다. 정렬 dance(후진/크리프)와 교차로 통과 전진은 거리로 동작하므로, 짝이 되는 PWM 을 바꾸면 시간도 다시 맞춰야 합니다:

- `MOTOR_POWER` ↑ → `REALIGN_BACKUP_MS`(후진)·`REALIGN_CREEP_MS`(크리프) 거리가 늘어 선을 지나침 → 시간 ↓.
- `MOTOR_POWER` ↓ → 크리프가 `MOTOR_POWER-40` 으로 구동되어 stall 위험. 후진 거리도 짧아짐.
- `DRIVE_END_PWM` 변경 → `CROSSING_PASS_MS`(마지막 교차로 통과) 거리가 변함.

각 매크로 위 주석의 ↔ 함께 확인 표시(`Settings_robot1.h:87` 부근)를 따라가면 한 값을 바꿀 때 같이 점검할 변수를 알 수 있습니다. 특히 `MOTOR_POWER`(cruise) 는 감속 바닥 `DRIVE_END_PWM`(역전 금지: `cruise > DRIVE_END_PWM > 0`)·PD 3종·정렬 dance·`base±correction<255` 와 모두 엮입니다.

8. 디버그 출력 활용

본 스케치는 9600 baud Serial 로 출력합니다 — Arduino IDE Serial Monitor (또는 `arduino-cli monitor -c baudrate=9600`) 로 보세요.

디버그 토글 (실주행/대회 전 정리):

매크로	기본	켜면
<code>DEBUG_TRACE</code> (<code>Settings_robot1.h:132</code>)	0	<code>LineTrace</code> 가 200ms마다 raw/정규화 출력. △ 핫 루프 Serial 블로킹으로 교차로 미인식 위험 — 센서 튜닝 시에만
<code>DEBUG_APPROACH_TONE</code> (<code>Settings_robot1.h:107</code>)	1	감속 전환(정속→감속) 순간 짧은 부저음(솔 G5). 감속 시작 타이밍 귀로 확인
<code>DEBUG_TURN_PAUSE_MS</code> (<code>Settings_robot1.h:241</code>)	0	매 회전 후 그만큼 정지 → 각도기 측정

상시 출력(코드에 박힌 것):

- 부팅 시 캘리브 값 + 로봇 식별: `rW=... lW=... rB=... lB=...`, `ROBOT_ID=N` `GRID=CxR` `WH=(x,y)`
- **NavLog 자동 dump** — 직전 트립의 Eval/DynBlock 흐름을 EEPROM 에서 읽어 출력 후 클리어 (`init`). 엔트리 수는 `NAVLOG_ENTRIES`(`Settings_robot1.h:292`).
- 네비게이션(**BFS 최단경로**): `Nav: (x1,y1) -> (x2,y2)`, 재계획마다 `NavLog Eval (x,y) hd=.. pathLen=N` (BFS 라 `conn0/afterBlk/fwd` 필드는 미사용=0).
- 우회/실패: `Dyn-blocked cell (x,y) count=N` (장애물 칸 동적 차단 등록), `Nav STUCK: no path (BFS) / guard exceeded / obstacle at start-pad exit`.
- 교차로 카운트: `LINE!!! :N`
- 회전 진입/이탈: `Enter Pivot turn Left / Leave Pivot turn Right` 등
- 장애물: `sensor front C/L/R : c / l / r` (CheckObstacle 매회), `Obstacle! Backing to prev node.`, `Obstacle! Reversing...`
- 미등록 도시: `Unknown city UID: [...]`, 우회 실패: `Forward nav failed - returning to warehouse with cargo.`

시리얼 입력 명령:

- `m` 또는 `p` → 현재 그리드 상태맵 출력(`PrintStatusMap`): `@`=현재위치 `#`=창고 `!`=동적장애물(IR) `x`=정적장애물 `C`=도시 `:`=격리경계 밖 `.`=빈 교차로. 주행 중엔 `navigateTo` 가 블로킹하므로 창고에서 다음 RFID 대기(유휴) 중에만 처리됨.

네비게이션 = BFS 최단경로: 매 교차로에서 현재→목표 최단경로를 다시 계산하고(`computeBfsPath`), 장애물(`DoLineTrace==false`)을 만나면 그 칸을 동적 차단에 넣고 즉시 재계획해 우회합니다. 구 greedy+DFS 백트래킹(`Dead-end ... backtracking` 로그)은 제거되었습니다. 격리 경계 `NAV_MIN_X`(로봇2=4)는 BFS 후보에서 서쪽 변을 빼는 방식으로 반영됩니다.

부저는 디버그 외에도 출발 멜로디(`PlayMelody "도-파-라"`), 교차로 통과 순간 "도"(C6), 도시 출발 직전 "도-도" 2회(C6 x2) 가 울립니다.

9. 마지막 체크리스트 — 회전이 자꾸 어긋날 때

1. 배터리 전압 충분한가? 낮으면 모터 토크 떨어져 회전 부족.
2. 바퀴·모터 마운트가 흔들리지 않는가? 슬립 발생 시 회전각 들쭉날쭉.
3. 바닥 마찰이 일정한가? 매끈한 바닥과 카펫에서 회전각 다름. 캘리브 환경과 운영 환경 일치.
4. 화물 적재(`_CARGO`)와 비적재 값을 각각 맞췄는가? 한쪽만 맞추면 다른 쪽이 어긋남 (§4.1).
5. 정밀 정렬은 `y=0`(창고)/`y=7`(도시)에서만 작동합니다 (`LineTracer_preciseRealign`). 중간 행에서는 미세 어긋남 누적 가능. `PRECISE_REALIGN_ENABLE=0` 으로 dance 자세를 끌 수도 있으나(§4.5) 그 두 행의 회전 정확도는 떨어집니다.
6. `TURN_RAMP_*` 를 만진 뒤 `*_DELAY_MS` 재보정했는가? 가감속이 회전각을 보웁니다 (§4.1).
7. `_motorCalibL/R` 이 `NaN / 0` 이 아닌가? 시작 시 Serial `rW lW rB lB` 첫 줄로 확인 (§6).
8. 실주행 전 `DEBUG_TRACE=0`, `DEBUG_TURN_PAUSE_MS=0` 확인 — 커진 채면 주행이 느려지거나 멈춤 (§8).